

# Consulta 19/12/2023

Dado un arreglo  $A$ , considere el problema especificado de la siguiente manera:

Const  $N : \text{Int}, A : \text{array}[0, N) \text{ of } \text{Int};$

Var  $res : \text{Bool};$

$\{P : N \geq 0\}$

S

$\{Q : res = \langle \exists i : 0 \leq i \leq N : \langle \sum j : 0 \leq j < i : (j+1) * A.j \rangle = 0 \rangle\}$

- Calcular el resultado para  $A = [-4, 2, 4, -3]$  usando la especificación. Justificar, enumerando todos los elementos del rango.
- Derivar un programa imperativo que resuelva este problema. Usar un solo ciclo.
- Opcional:** Ejecutar el programa para  $A = [-4, 2, 4, -3]$ . Justificar mostrando los cambios de estado.

Cambio en el enunciado: “= 1” en lugar de “= 0”.

Este programa calcula si hay algún segmento inicial cuya suma de los elementos multiplicados por sus posiciones + 1 da igual a 1.

b) **Derivación:** Cambio de constante por variable:

$INV \equiv res = \langle \exists i : 0 \leq i \leq \text{lim} : \text{sum}.i = 1 \rangle \wedge 0 \leq \text{lim} \leq N$  (done  $\text{sum}.i = \langle \sum \dots \rangle$ )

$B \equiv \text{lim} < N$

De esta manera vale  $INV \wedge \neg B \Rightarrow Q$ .

**Cuerpo del ciclo:** Probamos con la asignación “ $res, \text{lim} := E, \text{lim} + 1$ ”. Y vemos la wp:

$wp.(res, \text{lim} := E, \text{lim} + 1).INV$

$\equiv \{ \text{def. wp} \}$

$E = \langle \exists i : 0 \leq i \leq \text{lim} + 1 : \text{sum}.i = 1 \rangle \wedge \underline{0 \leq \text{lim} + 1 \leq N}$

$\equiv \{ \text{hip. varias} \}$

$E = \langle \exists i : \underline{0 \leq i \leq \text{lim} + 1} : \text{sum}.i = 1 \rangle$

$\equiv \{ \text{lógica} \}$

$E = \langle \exists i : 0 \leq i \leq \text{lim} \vee i = \text{lim} + 1 : \text{sum}.i = 1 \rangle$

$\equiv \{ \text{lógica, part. de rango y rango unitario} \}$

$E = \langle \underline{\exists i : 0 \leq i \leq \text{lim} : \text{sum}.i = 1} \rangle \vee \text{sum}(\text{lim} + 1) = 1$

$\equiv \{ \text{hip. y def. sum} \}$

$E = res \vee \langle \underline{\sum j : 0 \leq j < \text{lim} + 1 : (j+1) * A.j} \rangle = 1$

← observar esta sumatoria es:  $1 * A.0 + 2 * A.1 + \dots + (\text{lim} + 1) * A.\text{lim}$

**(PROBLEMA BORDE: el elemento “actual” que estamos mirando es  $A.\text{lim}$ , no queremos tener una hip. que hable de ese elemento porque todavía no lo recorrimos)**

$\equiv \{ \text{lógica, part. de rango y rango unitario} \}$

$E = res \vee ( \langle \underline{\sum j : 0 \leq j < \text{lim} : (j+1) * A.j} \rangle + (\text{lim} + 1) * A.\text{lim} ) = 1$

Me trabo, no puedo llegar a la H.I. Pero puedo fortalecer (agregar una hipótesis):

$$INV' \equiv INV \wedge \text{sum} = \langle \sum j : 0 \leq j < \text{lim} : (j + 1) * A.j \rangle$$

**Cuerpo del ciclo de nuevo:** Proponemos: “res, sum, lim := E, F, lim + 1”

$$\begin{aligned} & \text{wp.}(\text{res, sum, lim} := \text{E, F, lim} + 1).INV' \\ \equiv & \{ \text{def. wp} \} \\ & \underline{\text{E} = \langle \exists i : 0 \leq i \leq \text{lim} + 1 : \text{sum}.i = 1 \rangle \wedge 0 \leq \text{lim} + 1 \leq N} \wedge \\ & \text{F} = \langle \sum j : 0 \leq j < \text{lim} + 1 : (j + 1) * A.j \rangle \\ \equiv & \{ \text{mismos pasos} \} \\ & \text{E} = \text{res} \vee (\langle \sum j : 0 \leq j < \text{lim} : (j + 1) * A.j \rangle + (\text{lim} + 1) * A.\text{lim}) = 1 \wedge \\ & \text{F} = \langle \sum j : 0 \leq j < \text{lim} + 1 : (j + 1) * A.j \rangle \\ \equiv & \{ \text{nueva hip.} \} \\ & \underline{\text{E} = \text{res} \vee (\text{sum} + (\text{lim} + 1) * A.\text{lim}) = 1} \wedge \\ & \text{F} = \langle \sum j : 0 \leq j < \text{lim} + 1 : (j + 1) * A.j \rangle \\ \equiv & \{ \text{elijo } \mathbf{E = res} \vee (\mathbf{\text{sum} + (\text{lim} + 1) * A.\text{lim}}) = 1 \} \\ & \text{F} = \langle \sum j : 0 \leq j < \text{lim} + 1 : (j + 1) * A.j \rangle \\ \equiv & \{ \text{part. rango y rango unitario} \} \\ & \text{F} = \langle \sum j : 0 \leq j < \text{lim} : (j + 1) * A.j \rangle + (\text{lim} + 1) * A.\text{lim} \\ \equiv & \{ \text{hip. nueva} \} \\ & \text{F} = \text{sum} + (\text{lim} + 1) * A.\text{lim} \\ \equiv & \{ \text{elijo } \mathbf{F = \text{sum} + (\text{lim} + 1) * A.\text{lim}} \} \\ & \text{True} \end{aligned}$$

**Inicialización:** Ejercicio. Va a dar: “res, sum, lim := False, 0, 0”.

**Ejercicio de funcional:**

$$p.xs = \langle \forall \text{ as, b, bs} : xs = \text{as} ++ (\text{b} \blacktriangleright \text{bs}) : b = \text{sum.as} + \text{sum.bs} \rangle$$

Esta función me dice si todo elemento es igual a la suma del resto de los elementos.

**Ejemplo:** con  $xs = [1, 3, 2]$ . tengo tres términos:

$$b = 1, \text{as} = [], \text{bs} = [3, 2] : 1 = 0 + 3 + 2 ?? \text{ NO}$$

$$b = 3, \text{as} = [1], \text{bs} = [2] : 3 = 1 + 2 ?? \text{ SI}$$

$$b = 2, \text{as} = [1, 3], \text{bs} = [] : 2 = 1 + 3 + 0 ?? \text{ NO}$$

**Desafío:** dar una lista  $xs$  para la cual da True.

$$xs = []$$

$$xs = [0, 0, 0]$$

**Derivación:** Probamos por inducción en  $xs$ .

$$p.[] \doteq \dots$$

$$p.(x \blacktriangleright xs) \doteq \dots$$

### Paso inductivo:

$p.(x \blacktriangleright xs)$   
= { def. p }  
 $\langle \forall as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) : b = sum.as + sum.bs \rangle$   
= { 3ro excluido }  
 $\langle \forall as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge (as = [] \vee as \neq []) : b = sum.as + sum.bs \rangle$   
= { distributividad y part. de rango }  
 $\langle \forall as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = [] : b = sum.as + sum.bs \rangle$   
 $\wedge \langle \forall as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as \neq [] : b = sum.as + sum.bs \rangle$   
= { cambio de variable  $as \leftarrow a \blacktriangleright as$  }  
 $.. \wedge \langle \forall a, as, b, bs : x \blacktriangleright xs = (a \blacktriangleright as) ++ (b \blacktriangleright bs) \wedge (a \blacktriangleright as) \neq [] : b = sum.(a \blacktriangleright as) + sum.bs \rangle$   
= { prop. de listas }  
 $.. \wedge \langle \forall a, as, b, bs : x \blacktriangleright xs = (a \blacktriangleright as) ++ (b \blacktriangleright bs) : b = sum.(a \blacktriangleright as) + sum.bs \rangle$   
= { prop. de listas }  
 $.. \wedge \langle \forall a, as, b, bs : x = a \wedge xs = as ++ (b \blacktriangleright bs) : b = sum.(a \blacktriangleright as) + sum.bs \rangle$   
= { elim. de variable a }  
 $.. \wedge \langle \forall as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = sum.(x \blacktriangleright as) + sum.bs \rangle$   
= { def. sum }  
 $.. \wedge \langle \forall as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = x + sum.as + sum.bs \rangle$

Me trabo, no hay forma de llegar a la H.I. Debo generalizar:

$gp.n.xs = \langle \forall as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = n + sum.as + sum.bs \rangle$

gp generaliza a p ya que: “ $p.xs = gp.0.xs$ ” (ejercicio: demostrar).

Luego, puedo definir:

$p.xs \doteq gp.0.xs$

(al final p no era por inducción sino que se define en una sola línea)

### Derivación de gp:

#### Paso inductivo:

$gp.n.(x \blacktriangleright xs)$   
= { mismos pasos }  
 $.. \wedge \langle \forall as, b, bs : xs = as ++ (b \blacktriangleright bs) : b = n + x + sum.as + sum.bs \rangle$   
= { H.I. }  
 $.. \wedge gp.(n+x).xs$   
= { vemos lo otro }  
 $\langle \forall as, b, bs : x \blacktriangleright xs = as ++ (b \blacktriangleright bs) \wedge as = [] : b = n + sum.as + sum.bs \rangle$   
 $\wedge gp.(n+x).xs$   
= { elim. var as }  
 $\langle \forall b, bs : x \blacktriangleright xs = [] ++ (b \blacktriangleright bs) : b = n + sum.[] + sum.bs \rangle$   
 $\wedge gp.(n+x).xs$   
= { prop. listas }  
 $\langle \forall b, bs : x \blacktriangleright xs = b \blacktriangleright bs : b = n + sum.[] + sum.bs \rangle$   
 $\wedge gp.(n+x).xs$

$= \{ \text{prop. listas} \}$   
 $\langle \forall b, bs : x = b \wedge xs = bs : b = n + \text{sum.[]} + \text{sum.bs} \rangle$   
 $\wedge \text{gp.}(n+x).xs$   
 $= \{ \text{elim. var y rango unitario} \}$   
 $x = n + \text{sum.[]} + \text{sum.xs} \wedge \text{gp.}(n+x).xs$   
 $= \{ \text{def. sum.[]} \}$   
 $x = n + \text{sum.xs} \wedge \text{gp.}(n+x).xs$

Caso base: asdsa

$\text{gp.n.}[]$   
 $= \{ \text{def. gp} \}$   
 $\langle \forall as, b, bs : [] = as ++ (b \blacktriangleright bs) : b = n + \text{sum.as} + \text{sum.bs} \rangle$   
 $= \{ \text{prop. listas} \}$   
 $\langle \forall as, b, bs : \text{False} : b = n + \text{sum.as} + \text{sum.bs} \rangle$   
 $= \{ \text{rango vacío} \}$   
 True

Resultado final:

$p.xs \doteq \text{gp.0.xs}$

$\text{gp.n.}[] \doteq \text{True}$   
 $\text{gp.n.}(x \blacktriangleright xs) \doteq x = n + \text{sum.xs} \wedge \text{gp.}(n+x).xs$

$\text{sum.}[] \doteq 0$   
 $\text{sum.}(x \blacktriangleright xs) \doteq x + \text{sum.xs}$

Ejercicio de funcional 2:

$f.xs.y = \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge ys = as ++ cs : \#as \rangle$

“el segmento inicial en común de xs e ys más largo”

Derivación: Inducción en xs.

Probamos este esquema:

$f.[] .ys \doteq \dots$   
 $f.(x \blacktriangleright xs).ys \doteq \dots$

Caso base:

$f.[] .ys$   
 $= \{ \text{def. f} \}$   
 $\langle \text{Max } as, bs, cs : [] = as ++ bs \wedge ys = as ++ cs : \#as \rangle$   
 $= \{ \text{elimino } as = [] \text{ y } bs = [] \}$   
 $\langle \text{Max } cs : ys = cs : \#[ ] \rangle$   
 $= \{ \text{rango unitario} \}$   
 0

Paso inductivo:

$f.(x \blacktriangleright xs).ys$   
 $= \{ \text{def. } f \}$   
 $\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs \wedge ys = as ++ cs : \#as >$   
 $= \{ \text{3ro excluido} \}$   
 $\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs \wedge ys = as ++ cs \wedge (as = [] \vee as \neq []): \#as >$   
 $= \{ \text{distrib. y part. rango} \}$   
 $\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs \wedge ys = as ++ cs \wedge as = [] : \#as >$   
 $\text{max } \langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs \wedge ys = as ++ cs \wedge as \neq [] : \#as >$   
 $= \{ \text{en el de abajo, cambio var } as \leftarrow a \blacktriangleright as \}$   
 $\dots \text{max } \langle \text{Max } a, as, bs, cs : \underline{x \blacktriangleright xs = (a \blacktriangleright as) ++ bs} \wedge \underline{ys = (a \blacktriangleright as) ++ cs} \wedge (a \blacktriangleright as) \neq []$   
 $\quad \quad \quad : \#(a \blacktriangleright as) >$

Mejor frenamos y hacemos (sub)inducción en ys.

Cambiamos el esquema:

$f.[].ys \doteq \dots$   
 $f.(x \blacktriangleright xs).[ ] \doteq \dots$   
 $f.(x \blacktriangleright xs).(y \blacktriangleright ys) \doteq \dots$

Paso inductivo para xs + caso base para ys:

$f.(x \blacktriangleright xs).[ ]$   
 $= \{ \text{varios pasos} \}$   
 $0$

Paso inductivo para xs + paso inductivo para ys:

$f.(x \blacktriangleright xs).(y \blacktriangleright ys)$   
 $= \{ \text{mismos pasos} \}$   
 $\dots \text{max } \langle \text{Max } a, as, bs, cs : x \blacktriangleright xs = (a \blacktriangleright as) ++ bs$   
 $\quad \quad \quad \wedge y \blacktriangleright ys = (a \blacktriangleright as) ++ cs \wedge \underline{(a \blacktriangleright as) \neq []} : \#(a \blacktriangleright as) >$   
 $= \{ \text{prop. listas} \}$   
 $\dots \text{max } \langle \text{Max } a, as, bs, cs : \underline{x \blacktriangleright xs = (a \blacktriangleright as) ++ bs}$   
 $\quad \quad \quad \wedge y \blacktriangleright ys = (a \blacktriangleright as) ++ cs : \#(a \blacktriangleright as) >$   
 $= \{ \text{prop. listas} \}$   
 $\dots \text{max } \langle \text{Max } a, as, bs, cs : x = a \wedge xs = as ++ bs$   
 $\quad \quad \quad \wedge \underline{y \blacktriangleright ys = (a \blacktriangleright as) ++ cs} : \#(a \blacktriangleright as) >$   
 $= \{ \text{prop. listas} \}$   
 $\dots \text{max } \langle \text{Max } a, as, bs, cs : \underline{x = a} \wedge xs = as ++ bs$   
 $\quad \quad \quad \wedge y = a \wedge ys = as ++ cs : \#(a \blacktriangleright as) >$   
 $= \{ \text{elim. var } a (=x) \}$   
 $\dots \text{max } \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge y = x \wedge ys = as ++ cs : \underline{\#(x \blacktriangleright as)} >$   
 $= \{ \text{def. } \# \}$   
 $\dots \text{max } \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge y = x \wedge ys = as ++ cs : \#as + 1 >$   
 $= \{ \text{distrib. + con max} \}$   
 $\dots \text{max } \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge \underline{y = x} \wedge ys = as ++ cs : \#as > + 1 \}$ 

- Caso 1:  $y = x$ :  
 $= \{ \text{sust.} \}$

 $\dots \text{max } \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge \underline{\text{True}} \wedge ys = as ++ cs : \#as > + 1 \}$ 
 $= \{ \text{logica} \}$ 
 $\dots \text{max } \langle \text{Max } as, bs, cs : xs = as ++ bs \wedge ys = as ++ cs : \#as > + 1 \}$ 
 $= \{ \text{H.I.} \}$

```

...max ( f.xs.ys + 1 )
    = { la otra parte }
<Max as,bs,cs : x►xs = as ++ bs ∧ ys = as ++ cs ∧ as = [ ] : #as > max (f.xs.ys + 1)
    = { elim var. as }
<Max bs,cs : x►xs = bs ∧ ys = cs : #[ ] > max (f.xs.ys + 1)
    = { elim var. y rango unitario }
        #[ ] max (f.xs.ys + 1)
    = { def. # }
        0 max (f.xs.ys + 1)
    • Caso 2: y ≠ x:
        = { sust. }
...max (<Max as,bs,cs : xs = as ++ bs ∧ False ∧ ys = as ++ cs : #as > + 1 )
    = { logica }
...max (<Max as,bs,cs : False : #as > + 1 )
    = { rango vacío }
...max (-infinito + 1 )
    = { prop. -infinito }
...max (-infinito)
    = { vemos lo otro (mismos pasos que recién hicimos) }
        0 max (-infinito)
    = { resuelvo max }
        0

```

Resultado final:

```

f.[ ].ys ÷ 0
f.(x►xs).[ ] ÷ 0
f.(x►xs).(y►ys) ÷ ( y = x → 0 max (f.xs.ys + 1)
    □ y ≠ x → 0
)

```

## Ejercicio imperativo final 5/12/2023:

verdadero?

2. Considere el problema de, dado un arreglo, calcular la máxima distancia entre dos elementos del arreglo, esto es:

```

Const N : Int, A : array[0, N) of Int;
Var r : Int;
{P : N ≥ 0}
S
{Q : r = (Max i, j : 0 ≤ i < j < N : |A.i - A.j|)}

```

a) Calcular el resultado para  $A = [3, -2, 1, 0, -2]$ . Justificar, enumerando todos los elementos del rango.

b) Derivar un programa imperativo que resuelva este problema. El programa **debe recorrer una sola vez el arreglo** (sin ciclos anidados).

Cambio de constante por variable:

$$\text{INV} \equiv r = \langle \text{Max } i, j : 0 \leq i < j < \text{pos} : |A.i - A.j| \rangle \wedge 0 \leq \text{pos} \leq N$$

$$B \equiv \text{pos} < N$$

Luego,  $\text{INV} \wedge \neg B \Rightarrow Q$ .

**Cuerpo del ciclo:** proponemos: “ $r, \text{pos} := E, \text{pos} + 1$ ”.

$$\begin{aligned} & \text{wp.}(r, \text{pos} := E, \text{pos} + 1). \text{INV} \\ &= \{ \text{def. wp} \} \\ & E = \langle \text{Max } i, j : 0 \leq i < j < \text{pos} + 1 : |A.i - A.j| \rangle \wedge \underline{0 \leq \text{pos} + 1 \leq N} \\ &= \{ \text{hip. varias} \} \\ & E = \langle \text{Max } i, j : \underline{0 \leq i < j < \text{pos} + 1} : |A.i - A.j| \rangle \\ &= \{ \text{logica} \} \\ & E = \langle \text{Max } i, j : 0 \leq i < j < \text{pos} \vee (0 \leq i < j \wedge j = \text{pos}) : |A.i - A.j| \rangle \\ &= \{ \text{part. rango} \} \\ &= \{ \text{logica} \} \\ & E = \langle \underline{\text{Max } i, j : 0 \leq i < j < \text{pos} : |A.i - A.j|} \rangle \\ & \quad \text{max } \langle \text{Max } i, j : 0 \leq i < j \wedge j = \text{pos} : |A.i - A.j| \rangle \\ &= \{ \text{hip.} \} \\ & E = r \text{ max } \langle \text{Max } i, j : 0 \leq i < j \wedge \underline{j = \text{pos}} : |A.i - A.j| \rangle \\ &= \{ \text{elim. var } j \} \\ & E = r \text{ max } \langle \text{Max } i : 0 \leq i < \text{pos} : \underline{|A.i - A.\text{pos}|} \rangle \\ & \quad \leftarrow \text{ojo: no fortalecer! hay problema de borde.} \\ &= \{ \text{USO AYUDA A: } |Z| = Z \text{ max } (-Z) \} \\ & E = r \text{ max } \langle \text{Max } i : 0 \leq i < \text{pos} : (A.i - \underline{A.\text{pos}}) \text{ max } (- (A.i - A.\text{pos})) \rangle \\ &= \{ \text{USO AYUDA B: regla del término} \} \\ & E = r \text{ max } \langle \text{Max } i : 0 \leq i < \text{pos} : A.i - \underline{A.\text{pos}} \rangle \\ & \quad \text{max } \langle \text{Max } i : 0 \leq i < \text{pos} : - (A.i - \underline{A.\text{pos}}) \rangle \\ &= \{ \text{aritmetica} \} \\ & E = r \text{ max } \langle \text{Max } i : 0 \leq i < \text{pos} : A.i + (-\underline{A.\text{pos}}) \rangle \\ & \quad \text{max } \langle \text{Max } i : 0 \leq i < \text{pos} : -A.i + \underline{A.\text{pos}} \rangle \\ &= \{ \text{USO AYUDA D: distrib. + con Max} \} \\ & E = r \text{ max } (\langle \underline{\text{Max } i : 0 \leq i < \text{pos} : A.i} \rangle + (-\underline{A.\text{pos}})) \\ & \quad \text{max } (\langle \underline{\text{Max } i : 0 \leq i < \text{pos} : -A.i} \rangle + \underline{A.\text{pos}}) \end{aligned}$$

Me trabo, pero veo que puedo hacer dos fortalecimientos:

$$\begin{aligned} \text{INV}' &= \text{INV} \wedge \text{max1} = \langle \text{Max } i : 0 \leq i < \text{pos} : A.i \rangle \\ & \quad \wedge \text{max2} = \langle \text{Max } i : 0 \leq i < \text{pos} : -A.i \rangle \end{aligned}$$

**Cuerpo del ciclo de nuevo:**

$$\begin{aligned} & \text{wp.}(r, \text{max1}, \text{max2}, \text{pos} := E, F, G, \text{pos} + 1) \\ &= \{ \text{mismos pasos} \} \\ & E = r \text{ max } (\langle \underline{\text{Max } i : 0 \leq i < \text{pos} : A.i} \rangle + (-A.\text{pos})) \\ & \quad \text{max } (\langle \underline{\text{Max } i : 0 \leq i < \text{pos} : -A.i} \rangle + A.\text{pos}) \\ & \wedge F = \langle \text{Max } i : 0 \leq i < \text{pos} + 1 : A.i \rangle \end{aligned}$$

$\wedge G = \langle \text{Max } i : 0 \leq i < \text{pos}+1 : -A.i \rangle$   
 $= \{ \text{hip. nuevas} \}$   
 $E = r \text{ max } ( \text{max1} + (-A.\text{pos}) ) \text{ max } ( \text{max2} + A.\text{pos} )$   
 $\wedge F = \langle \text{Max } i : 0 \leq i < \text{pos}+1 : A.i \rangle$   
 $\wedge G = \langle \text{Max } i : 0 \leq i < \text{pos}+1 : -A.i \rangle$   
 $= \{ \text{elijo } E = r \text{ max } ( \text{max1} + (-A.\text{pos}) ) \text{ max } ( \text{max2} + A.\text{pos} ) \}$   
 $F = \langle \text{Max } i : 0 \leq i < \text{pos}+1 : A.i \rangle$   
 $\wedge G = \langle \text{Max } i : 0 \leq i < \text{pos}+1 : -A.i \rangle$   
 $= \{ \text{pasos varios} \}$   
 $F = \text{max1 max } A.\text{pos}$   
 $\wedge G = \text{max2 max } (-A.\text{pos})$   
 $= \{ \text{elijo convenientemente} \}$   
 True

**Resultado final:**

Const ...

Var ...

r, max1, max2, pos := -infinito, -infinito, -infinito, 0 ;

**do** pos < N  $\rightarrow$

    r, max1, max2, pos :=

        r max ( max1 + (-A.pos) ) max ( max2 + A.pos ),

        max1 max A.pos,

        max2 max (- A.pos),

        pos+1

**od**